

ML Beginner's Revision Notes

Exploratory Data Analysis (EDA) & Feature Engineering Practical Guide

Pandas

Regex Cleaning

Missing Value Imputation

One-Hot Encoding

Seaborn / Matplotlib

CORE ML CONCEPT

What is EDA & Feature Engineering?

Before training any Machine Learning model, **80% of a Data Scientist's time** is spent preparing data. Raw real-world data is messy—it contains non-numeric symbols, missing values, incorrect data types, and text strings that ML algorithms cannot directly compute. **Exploratory Data Analysis (EDA)** helps us understand data patterns, while **Feature Engineering** transforms raw inputs into clean numerical representations required by algorithms.

Step 1: Environment Setup & Data Loading

PRACTICAL CODE

1.1 Importing Required Libraries

We start by importing fundamental Python libraries: pandas for tabular data manipulation, numpy for numerical operations, and matplotlib / seaborn for visual analysis.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings # Suppress warning messages for clean notebook display
warnings.filterwarnings('ignore')
```

1.2 Loading the Excel File

Reading an Excel spreadsheet into a Pandas DataFrame using `pd.read_excel()`.

```
# Load dataset from Excel file
emp = pd.read_excel(r"D:\DataScience_Prakash_Senapathi\EDA_Practical1.xlsx")
```

Beginner Pitfall: If you get `ImportError: Missing optional dependency 'openpyxl'` when running `read_excel`, install it using `pip install openpyxl` in your terminal.

Step 2: Initial Data Inspection & Health Check

DATA AUDIT

2.1 Inspecting Dataset Dimensions, Columns, and Preview

Always inspect the raw structure before applying transformations.

```
# Check column names and dimension shape (rows, columns) print(emp.columns) print(emp.shape) # Returns tuple: (6, 6) #
Inspect top 5 and bottom 5 rows emp.head() emp.tail()
```

Initial Raw Data Snapshot:

Index	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	NaN	NaN	15000	4
3	Jane	Analytics	NaN	Hyderabad	20000	NaN
4	Uttam	Statistics	67	NaN	30000	5
5	Kim	NLP	55	Delhi	60000	10

2.2 Checking Structural Integrity & Missing Values

```
# Concise summary of dtypes and non-null counts emp.info() # Count missing values per column emp.isnull().sum()
```

Missing Values Count:

Name: 0 | Domain: 0 | Age: 2 | Location: 2 | Salary: 0 | Exp: 1

Data Types:

All 6 columns are stored as generic 'object' (string) dtype due to dirty formatting!

Key takeaway: Notice that even numerical columns like Age, Salary, and Exp are classified as object. This happens when raw data contains dirty symbols, unneeded formatting, or strings.

Step 3: Data Cleaning & Text Preprocessing (Regex)

STRING MANIPULATION

3.1 Removing Non-Alphanumeric Characters (\W)

Non-alphanumeric characters (like hidden spaces, symbols, punctuation) cause string match failures. We clean the Name and Salary columns using Regular Expressions (Regex).

```
# Regex '\W' matches any non-word character (anything except a-z, A-Z, 0-9, _) emp['Name'] =
emp['Name'].str.replace(r'\W', "", regex=True) emp['Salary'] = emp['Salary'].str.replace(r'\W', "", regex=True)
```

3.2 Extracting Numeric Values (\d+)

If experience strings contain stray text or formatting, we extract only the digit sequences.

```
# Regex '(\d+)' extracts one or more consecutive numerical digits emp['Exp'] = emp['Exp'].str.extract('(\d+)')
```

3.3 Creating a Clean Dataset Copy

Always preserve the raw dataset and work on an explicit deep copy to prevent corrupting original source data.

```
clean_data = emp.copy()
```

Step 4: Handling Missing Data (Imputation)

CRUCIAL STEP

4.1 Numerical Imputation using Mean

Machine Learning models cannot handle NaN (Not a Number) values. For numerical features, a standard approach is replacing missing values with the column **Mean (Average)**.

```
# Convert column to numeric, compute mean, and fill NaN values mean_age = pd.to_numeric(clean_data['Age']).mean() # (34 + 45 + 67 + 55) / 4 = 50.25 clean_data['Age'] = clean_data['Age'].fillna(mean_age) mean_exp = pd.to_numeric(clean_data['Exp']).mean() # (2 + 3 + 4 + 5 + 10) / 5 = 4.8 clean_data['Exp'] = clean_data['Exp'].fillna(mean_exp)
```

4.2 Categorical Imputation using Mode

For categorical/text columns like Location, taking an average is mathematically impossible. We fill missing values with the **Mode (Most Frequent Value)**.

```
# Find mode and fill NaN values in categorical column mode_loc = clean_data['Location'].mode()[0] # 'Bangalore' clean_data['Location'] = clean_data['Location'].fillna(mode_loc)
```

Common Error Analysis: In the original document code, line 139 threw an `AttributeError: module 'numpy' has no attribute 'mode'` when running `np.mode()`.

Explanation: NumPy does NOT have a top-level `.mode()` method! Mode is obtained via Pandas series `clean_data['Location'].mode()[0]` or `scipy.stats.mode()`.

Step 5: Type Casting (Data Type Conversion)

TYPE OPTIMIZATION

5.1 Converting Object Types to Proper Dtypes

After imputation, numbers are still stored as generic strings or floats. We convert numeric features to integers and text strings to categorical types using `.astype()`.

```
# Convert numeric columns to Integer clean_data['Age'] = clean_data['Age'].astype(int) clean_data['Exp'] = clean_data['Exp'].astype(int) clean_data['Salary'] = clean_data['Salary'].astype(int) # Convert text columns to Category for visualization & memory efficiency clean_data['Name'] = clean_data['Name'].astype('category') clean_data['Domain'] = clean_data['Domain'].astype('category') clean_data['Location'] = clean_data['Location'].astype('category')
```

Post-Cleaning Dataset State:

Index	Name	Domain	Age	Location	Salary	Exp
0	Mike	Datascience	34	Mumbai	5000	2

Index	Name	Domain	Age	Location	Salary	Exp
1	Teddy	Testing	45	Bangalore	10000	3
2	Umar	Dataanalyst	50	Bangalore	15000	4
3	Jane	Analytics	50	Hyderbad	20000	4
4	Uttam	Statistics	67	Bangalore	30000	5
5	Kim	NLP	55	Delhi	60000	10

Step 6: Data Visualization with Seaborn

VISUAL ANALYTICS

6.1 Understanding Distributions and Relationships

Visualizing data allows us to identify skewness, trends, and outliers.

```
# Univariate Distribution Plot (Histogram + KDE) sns.histplot(clean_data['Salary'], kde=True) plt.title('Salary Distribution')
plt.show() # Linear Regression Trend Plot (Bivariate Relationship) sns.lmplot(data=clean_data, x='Exp', y='Salary')
plt.title('Experience vs Salary Trend') plt.show()
```

Common Error Analysis: In the document code (Cell 159 & 160), calling `vis1.show()` or `vis2.show()` threw an `AttributeError` because Seaborn plot objects (Axes / FacetGrid) do not have a `.show()` method.
Correct Approach: Always use `plt.show()` from `matplotlib.pyplot` to render figure plots!

Step 7: Categorical Encoding (One-Hot Encoding)

ML PIPELINE READY

7.1 Why One-Hot Encoding is Required

Machine Learning models perform mathematical matrix operations (e.g., $y = W \cdot X + b$). Text labels like "Mumbai" or "Datascience" cannot be multiplied in linear algebra. We convert categorical variables into **binary dummy variables (0 or 1)** using `pd.get_dummies()`.

```
# Convert categorical columns into numerical binary indicators encoded_data = pd.get_dummies(clean_data, dtype=int)
```

Sample Encoded Features Matrix Output:

Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Domain_Datascience	Location_Bangalore
34	5000	2	0	0	1	1	0
45	10000	3	0	0	0	0	1
50	15000	4	0	0	0	0	1
50	20000	4	1	0	0	0	0

Age	Salary	Exp	Name_Jane	Name_Kim	Name_Mike	Domain_Datascience	Location_Bangalore
67	30000	5	0	0	0	0	1
55	60000	10	0	1	0	0	0

Result: Every single column in `encoded_data` is now 100% numerical (integers/floats) with zero missing values—ready to be fed into algorithms like Linear Regression, Decision Trees, or Random Forests!

Quick Revision Cheat Sheet for Beginners

1. Data Inspection

`df.head()` - Top 5 rows
`df.info()` - Column types & non-nulls
`df.isnull().sum()` - Missing values count

2. Regex Cleaning

`r'\W'` - Non-alphanumeric chars
`r'\d+'` - Extract numeric digits
`df.copy()` - Create deep copy

3. Imputation Rules

Numeric: Mean/Median fill
Categorical: Mode fill
`df[col].fillna(val)`

4. Encoding & Prep

`df.astype(int)` - Type casting
`pd.get_dummies()` - One-Hot Encoding
`plt.show()` - Render Seaborn plots